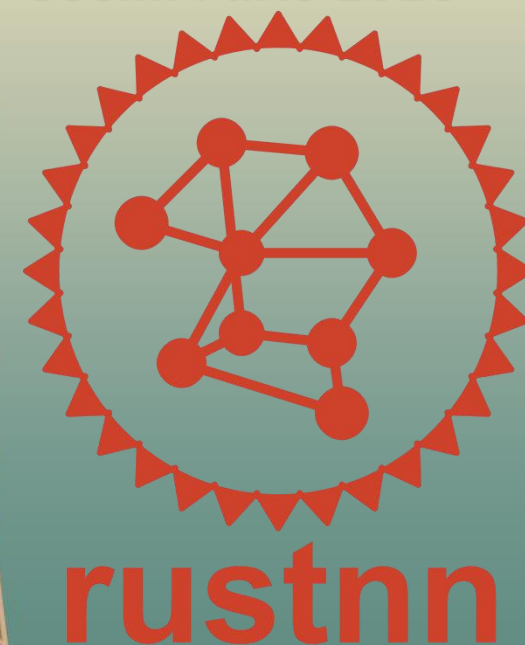


RustNN: A Rust Ecosystem for the Web Neural Network Standard

Markus Tavenrath · GoSim Paris 2026



WebNN: One Graph → Any Hardware

WebNN is the W3C standard for neural network inference

- You describe your computation via MLGraphBuilder.
- The Browser compiles and dispatches it to the best hardware.
 - CPU, GPU, NPU — same graph*
 - Browser delegates to Windows ML ▪ CoreML ▪ LiteRT
- Origin trials started in Chromium and Edge

One graph builder. The platform handles the rest.
But it's browser-only.

executing-an-MLGraph-using-MLTensors.js

```
1 const descriptor = {dataType: 'float32', shape: [2, 2]};
2 const context = await navigator.ml.createContext();
3 const builder = new MLGraphBuilder(context);
4
5 // 1. Create a computational graph 'C = 0.2 * A + B'.
6 const constant = builder.constant(descriptor, new Float32Array(4).fill(0.2));
7 const A = builder.input('A', descriptor);
8 const B = builder.input('B', descriptor);
9 const C = builder.add(builder.mul(A, constant), B);
10
11 // 2. Compile the graph.
12 const graph = await builder.build({C: C});
13
14 // 3. Create reusable input and output tensors.
15 const [inputTensorA, inputTensorB, outputTensorC] =
16   await Promise.all([
17     context.createTensor({
18       dataType: A.dataType, shape: A.shape, writable: true
19     }),
20     context.createTensor({
21       dataType: B.dataType, shape: B.shape, writable: true
22     }),
23     context.createTensor({
24       dataType: C.dataType, shape: C.shape, readable: true
25     })
26   ]);
27
28 // 4. Initialize the inputs.
29 context.writeTensor(inputTensorA, new Float32Array(4).fill(1.0));
30 context.writeTensor(inputTensorB, new Float32Array(4).fill(0.8));
31
32 // 5. Execute the graph.
33 const inputs = {
34   'A': inputTensorA,
35   'B': inputTensorB
36 };
37 const outputs = {
38   'C': outputTensorC
39 };
40 context.dispatch(graph, inputs, outputs);
41
42 // 6. Read back the computed result.
43 const result = await context.readTensor(outputTensorC);
44 console.log('Output value:', new Float32Array(result)); // [1, 1, 1, 1]
```

Source: <https://webnn.io/en/learn/get-started/quickstart>



WebNN: One Graph $\rightarrow$ Any Hardware

Why not just WebGPU or WebAssembly?

- With WebGPU/WASM, getting something functional is straightforward
- But browsers run on every device with a screen — desktop, mobile, low-power, NPU-equipped
- Getting performance portability across that entire landscape is a different problem entirely
 - NPUs are not supported at all
- WebNN delegates that to the platform: vendor-tuned kernels, NPU scheduling — handled by the OS, not you



WebNN Samples

continuously update based on WebNN spec

Tensors

A = [[1,2],[2,4]]
B = [[3,3],[5,2]]
C = matmul(A,B)
Expect: [[38,17],[26,14]]
T = [[1,2],[3]] # invalid
H = [[2,3],[5,4,1]]
W = [[4,1],[6,3],[2,4]]

WebNN Playground

A browser-based playground for experimenting with WebNN expressions without downloading code.

WebNN Code Editor

Evaluate, review and modify WebNN sample codes interactively in web browser.

Image Classification

Classifying the major object in the image into a set of pre-defined classes.

Handwritten Digits Classification

Use LeNet to classify handwritten digits from the classic MNIST example data set.

Fast Style Transfer

Applying the STAFFNet architecture and other painting styles of Van Gogh to your image.

Noise Suppression (NSNet2)

A NSNet2 baseline implementation of deep learning-based noise suppression.

Noise Suppression (TNNNoise)

An TNNNoise baseline implementation of deep learning-based noise suppression.

Object Detection

Detecting instances of semantic objects of a certain class in digital images and videos.

Selfie Segmentation

Segment the prominent humans in the scene.

Semantic Segmentation

Partitioning image into semantically meaningful parts to classify each part into pre-determined class.

Facial Landmark Detection

Detecting facial landmarks like eyes, nose, mouth, etc, which can be used for web-based try-on simulator of online stores.

Face Recognition

Detecting faces of participants using object detection, and checking whether each face was present or not.

The WebNN API is under active development within W3C Web Machine Learning Working Group, please file a bug report if the WebNN sample doesn't work in the latest versions of Chrome or Edge.

©2024 WebNN API - Installation Guides - Implementation Status

<https://webmachinelearning.github.io/webnn-samples/>

WebNN Developer Preview

Run ONNX models in the browser with WebNN. The developer preview unlocks interactive ML on the web that benefits from reduced latency, enhanced privacy and security, and GPU acceleration from Windows ML. To run the samples below, please ensure the WebNN flag is enabled in about:flags. For more information on how to get started with the WebNN Developer Preview, please visit our documentation.

Stable Diffusion 1.5

Text-to-Image

Stable Diffusion Turbo

Text-to-Image

Stable Diffusion XL Turbo

Text-to-Image

Segment Anything

Image Segmentation

Whisper Base

Automatic Speech Recognition

Image Classification

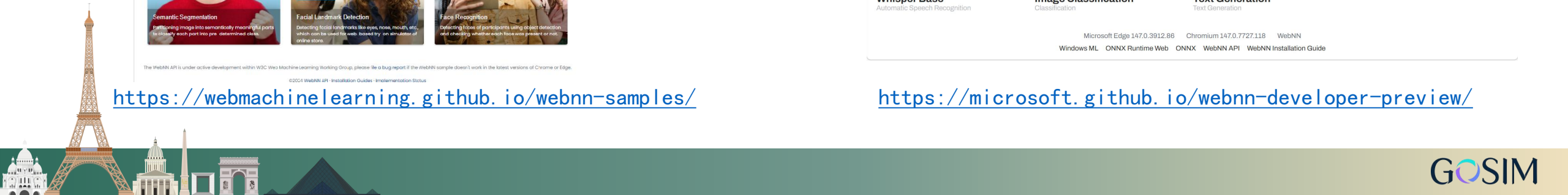
Classification

Text Generation

Text Generation

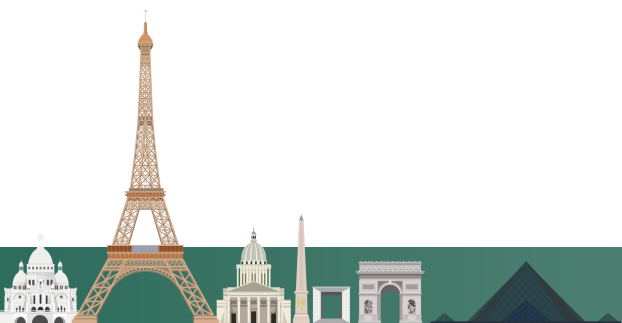
Microsoft Edge 147.0.3912.86 Chromium 147.0.7727.118 WebNN
Windows ML ONNX Runtime Web ONNX WebNN API WebNN Installation Guide

<https://microsoft.github.io/webnn-developer-preview/>



Real workloads which can be targeted by WebNN

v2	Video conferencing	– concurrent ML, audio/video, background removal, noise removal
	Object detection	– YOLO12n (real-time)
	Vision	– MODNet ▪ Segment Anything ▪ Depth Anything
	Image generation	– Stable Diffusion Turbo
	Speech	– Recognition: Whisper ▪ Generation: Kokoro ▪ Piper
	On-device SLMs	– Qwen2.5 ▪ gpt2 ▪ llama-2-7b



If WebNN works great in the browser, can it work well on the desktop as well?

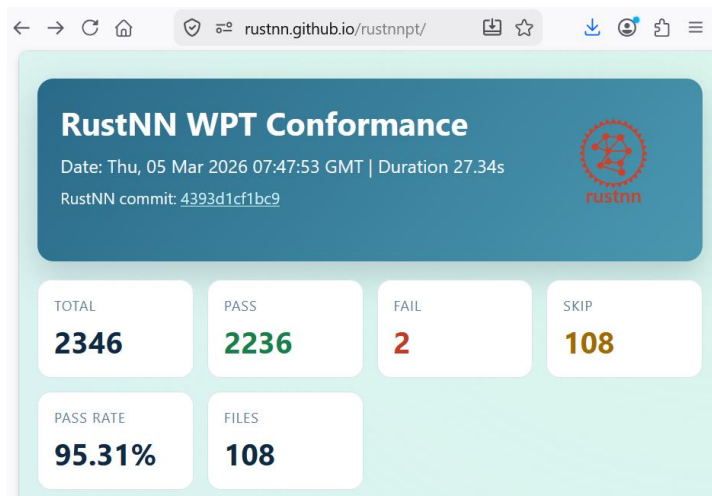
If it works well, can we build a shared ecosystem?



- Chromium has a pretty good WebNN implementation on top of ONNX Runtime & LiteRT
- Unlike WebGPU (Dawn) their implementation is deeply intertwined with Chromium
- What about other browsers or native applications?
 - [rustnn – a Python and Rust Implementation of W3C WebNN aimed at Firefox | Tarek Ziadé](#)
 - Work started around December 2025.
- I joined the project in January after realizing its potential
 - A native implementation would allow for a single deployment path in the web and desktop
 - It is designed to run on any other important AI Inference API
- A full ecosystem flow is required to get the developer where he is today
 - Python!



- RustNN core
 - Everything you need to implement the WebNN standard
- Tested through rustnnt, a rust wrapper to run the WebNN WPT Tests
 - A subset of the tests does not run yet due to the bindings



RustNN
Core
Graph Representation & Validation

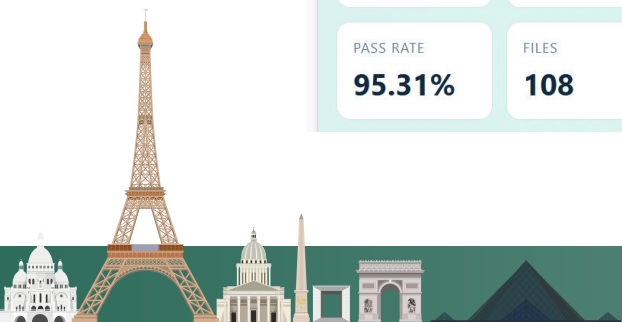
Conversion & Execution

ORT

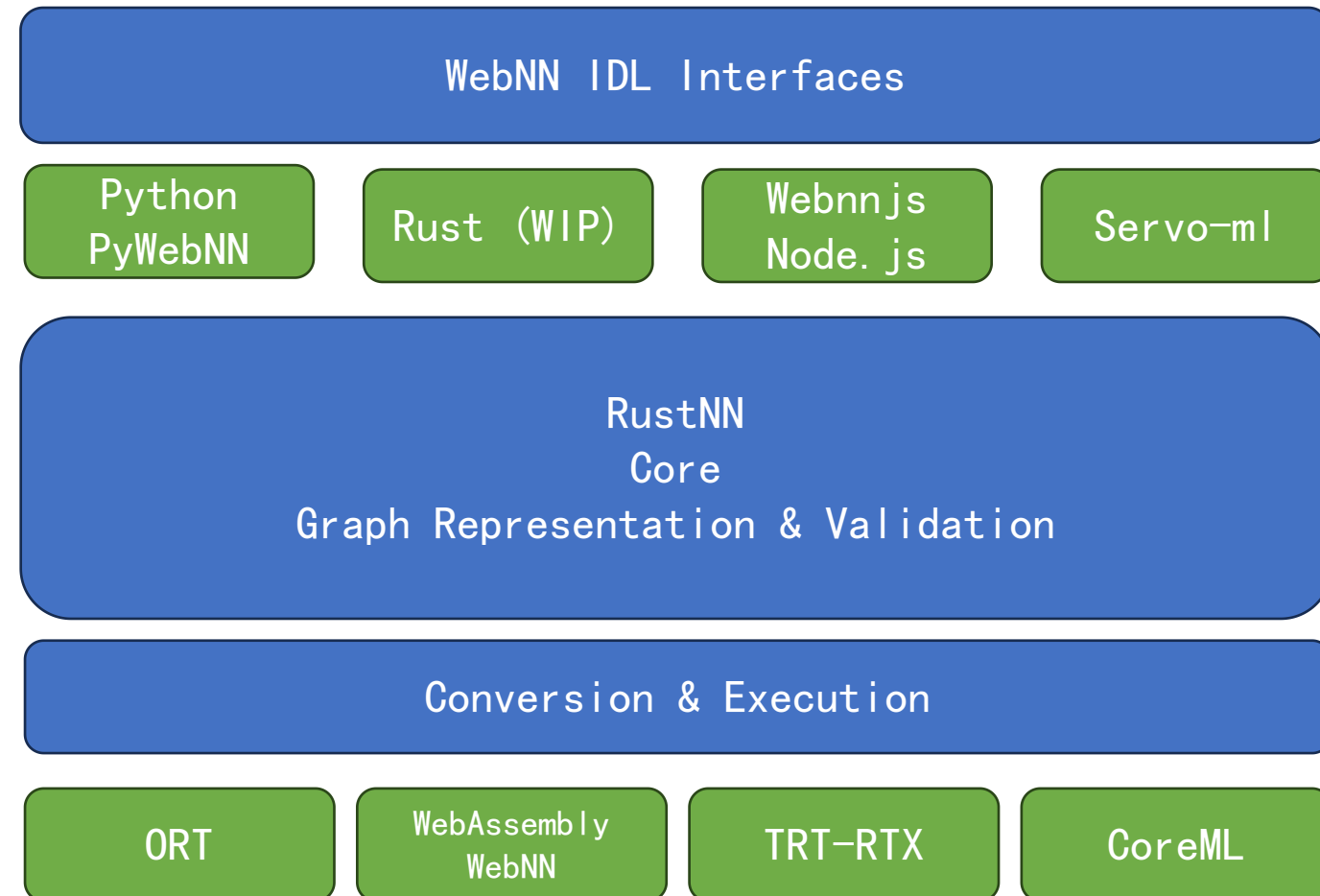
WebAssembly
WebNN

TRT-RTX

CoreML



- WebNN IDL language bindings
 - Implemented on top of RustNN
 - Python, Node.js are fine
 - Rust is work in progress
 - Servo-ml is a servo fork
 - Adds WebNN support
 - C/C++ are under planning



PyWebNN

- Develop network in Python
- Use Torch / Numpy to verify math
- Once satisfied call `graph.save(“graph.webnn”)`

```
import numpy as np
import webnn

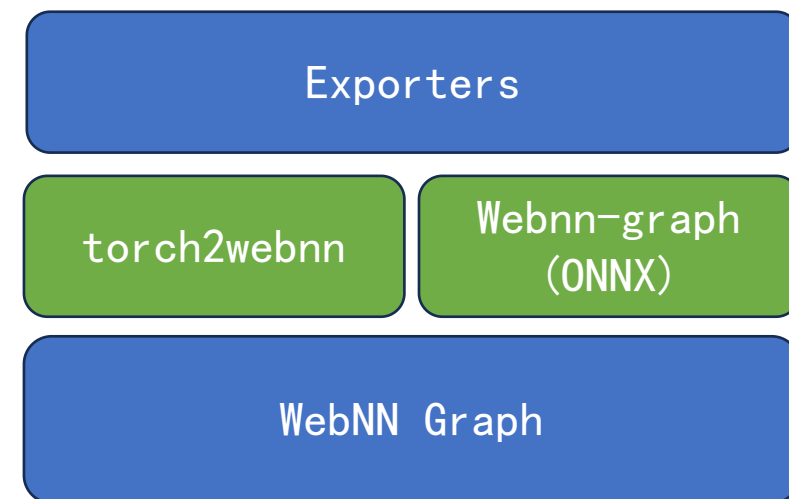
ml = webnn.ML()
context = ml.create_context(device_type="cpu")
builder = context.create_graph_builder()

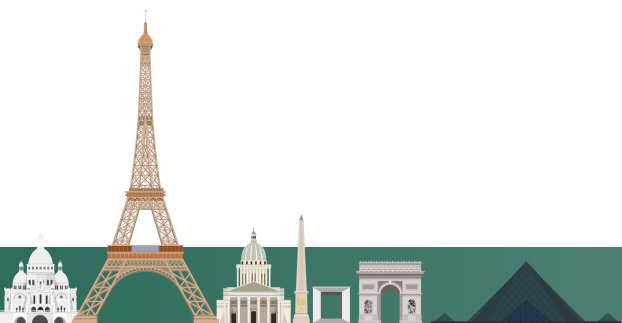
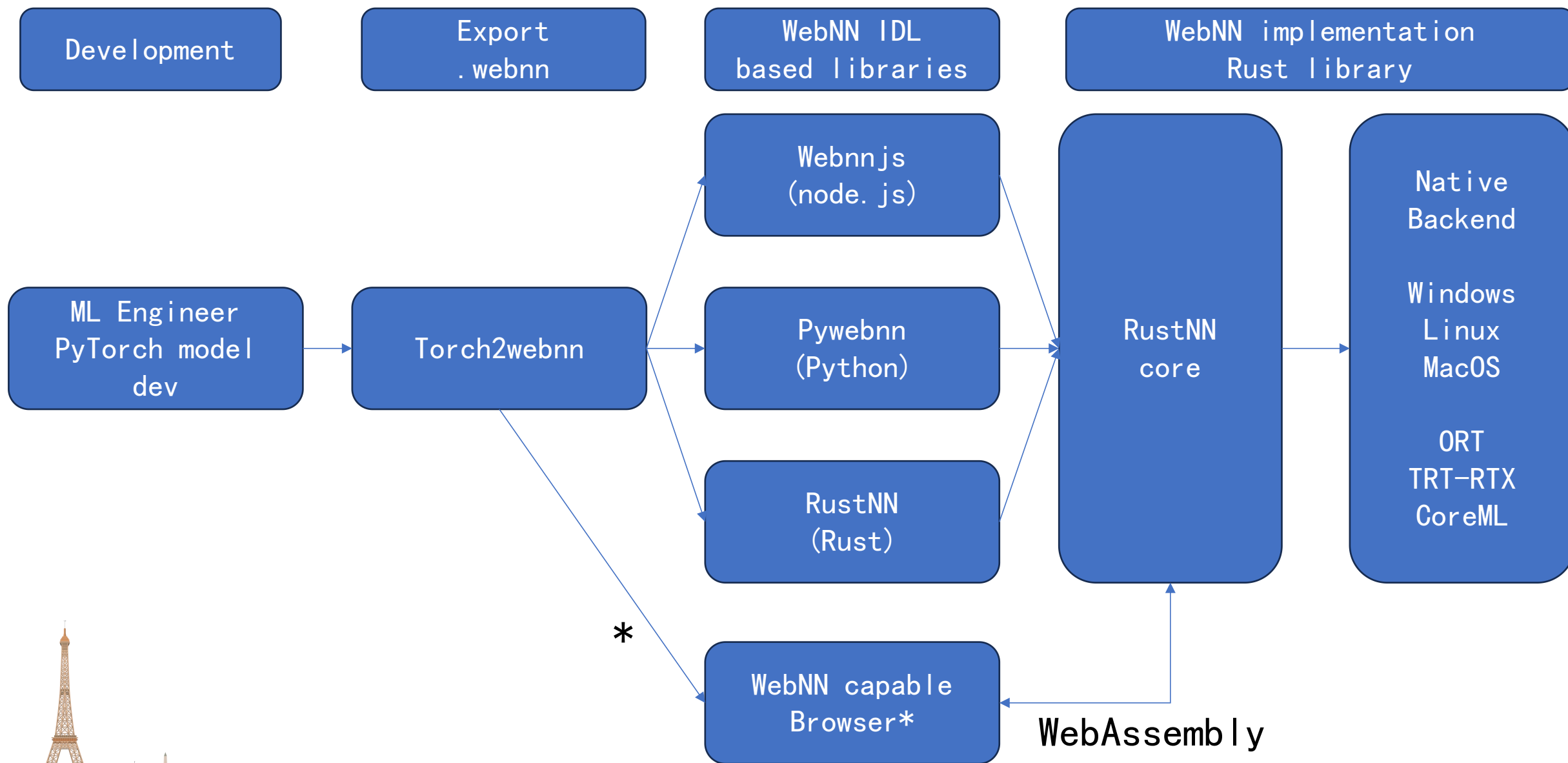
x = builder.input("x", [2, 3], "float32")
y = builder.input("y", [2, 3], "float32")
out = builder.relu(builder.add(x, y))
graph = builder.build({"output": out})

result = context.compute(
    graph,
    {
        "x": np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float32),
        "y": np.array([[0.1, 0.2, 0.3], [0.4, 0.5, 0.6]], dtype=np.float32),
    },
)
print(result["output"])
```



- How do we get content into the ecosystem?
 - RustNN ecosystem should be fully self-contained.
- Required projects
 - webnn-graph
 - A graph DSL & rust-based library to load and save the DSL.
 - Designed to be close to the WebNN graph
 - Can use .safetensors for weights
 - onnx2webnn (currently part of webnn-graph)
 - Early phase, only basic models supported
 - Goal: Support broad ONNX ecosystem
 - torch2webnn
 - Early phase, used to export demo model
 - Converts Aten IR to webnn-graph
 - Goal: As lossless as possible PyTorch → WebNN export







What's Next

- Conformance
 - Ensure we can run 100% of the conformance tests for better test coverage
- Browser integration
 - Update Firefox integration prototype
 - This gives us more content for testing and benchmarks
- Generic GPU Backend?
 - WebGPU has experimental support for subgroup matrices (TensorCores)
 - Supported by Vulkan, Metal and D3D12 though LinAlg Matrix Preview
 - Use ORT-WebGPU or Burn? Burn is more flexible (CPU, Vulkan, CUDA, Metal, ...)
- Content!
 - Export more models through torch2webnn
 - Harden the exporter and backends



Get Involved

GitHub: <http://github.com/rustnn>
Try it: `pip install pywebnn` — experiment with WebNN in the python ecosystem
Test it: `run rustnnt conformance suite`
Contribute: file issues, PRs and most important
Use it: Develop networks and demos to show what is possible with WebNN



WebNN: one graph → any hardware — but locked in the browser.

RustNN brings WebNN to the edge: standalone · embeddable · spec-compliant · no browser required.

Train in PyTorch, deploy anywhere — CPU, GPU, NPU, any language, any device.

github.com/rustnn

Questions?

